Name:Khushi Singh
Class:D20C Roll no.34

Experiment No.10

Title: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Aim: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Prerequisites:

- Windows Subsystem for Linux (WSL 2) with Ubuntu 22.04.5 LTS (kernel 6.6.87.2-microsoft-standard-WSL2)
- Docker Engine 29.1.3 (docker.io, running inside the Ubuntu WSL distribution)
- Git 2.34.1, jq 1.6, curl
- Node.js v18.20.8 and npm 10.8.2

Theory:

1. Hyperledger Fabric

Hyperledger Fabric is an open-source, **permissioned** (private) blockchain framework hosted by the Linux Foundation. Unlike public blockchains, every participant has a known identity issued by a Membership Service Provider (MSP). There is no cryptocurrency or mining: transactions are endorsed by trusted peers and ordered by an ordering service. This makes Fabric suited to enterprise use cases such as supply chains, asset tracking and finance.

2. Network components

- **Peers:** nodes that hold a copy of the ledger (blockchain + world state) and run chaincode. The test network has *peer0.org1.example.com* (port 7051) and *peer0.org2.example.com* (port 9051).
- **Orderer:** *orderer.example.com* (port 7050) puts endorsed transactions into blocks using Raft consensus and sends the blocks to the peers.
- **MSP / certificates:** identities for Org1, Org2 and the orderer organisation, generated with *cryptogen* (or a Fabric CA). TLS is used for all connections.
- **Channel:** a private sub-ledger (*mychannel*) shared only by its member organisations.

3. Chaincode and its lifecycle

Chaincode is Fabric's smart contract. The *asset-transfer-basic* JavaScript chaincode stores assets (ID, Color, Size, Owner, AppraisedValue) and provides InitLedger, CreateAsset, ReadAsset, UpdateAsset, DeleteAsset, TransferAsset and GetAllAssets. The Fabric 2.x lifecycle is: **package → install** on each peer → **approve** by each organisation → **commit** the definition to the channel.

4. Transaction flow (execute – order – validate)

A client sends a proposal to the endorsing peers, which simulate the chaincode and sign the result. The client submits the endorsed transaction to the orderer, which adds it to a block. Every peer

validates the endorsement policy and commits the block. A *query* is read-only and is answered by one peer without creating a transaction, while an *invoke* changes the ledger and is committed with status VALID.

Procedure:

Step 1: Update the System

Command:

```
sudo apt update
sudo apt upgrade -y
```

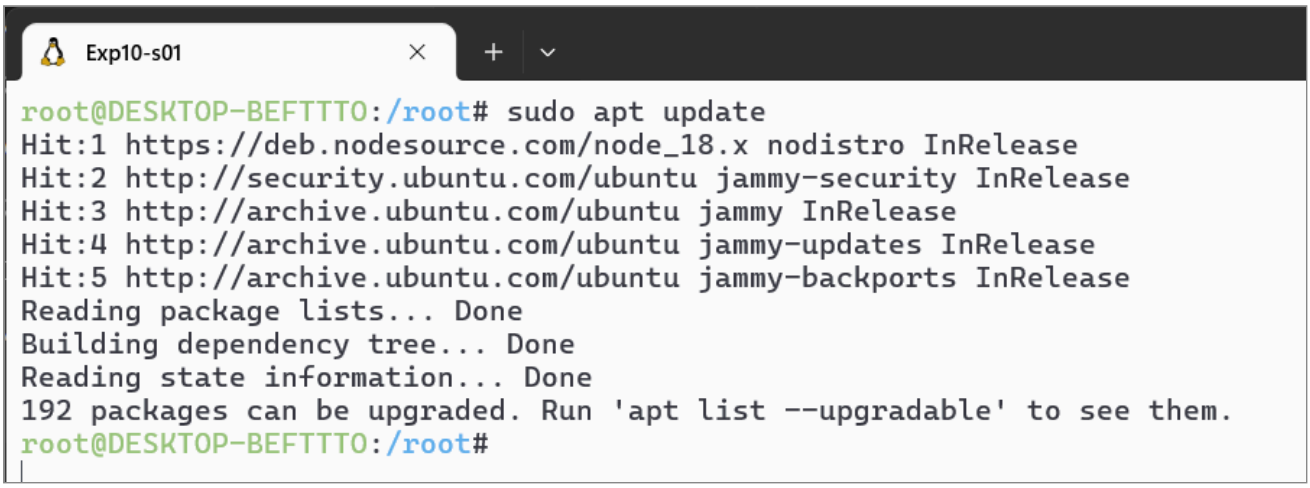


Fig. 1: sudo apt update – package lists refreshed (192 packages upgradable)

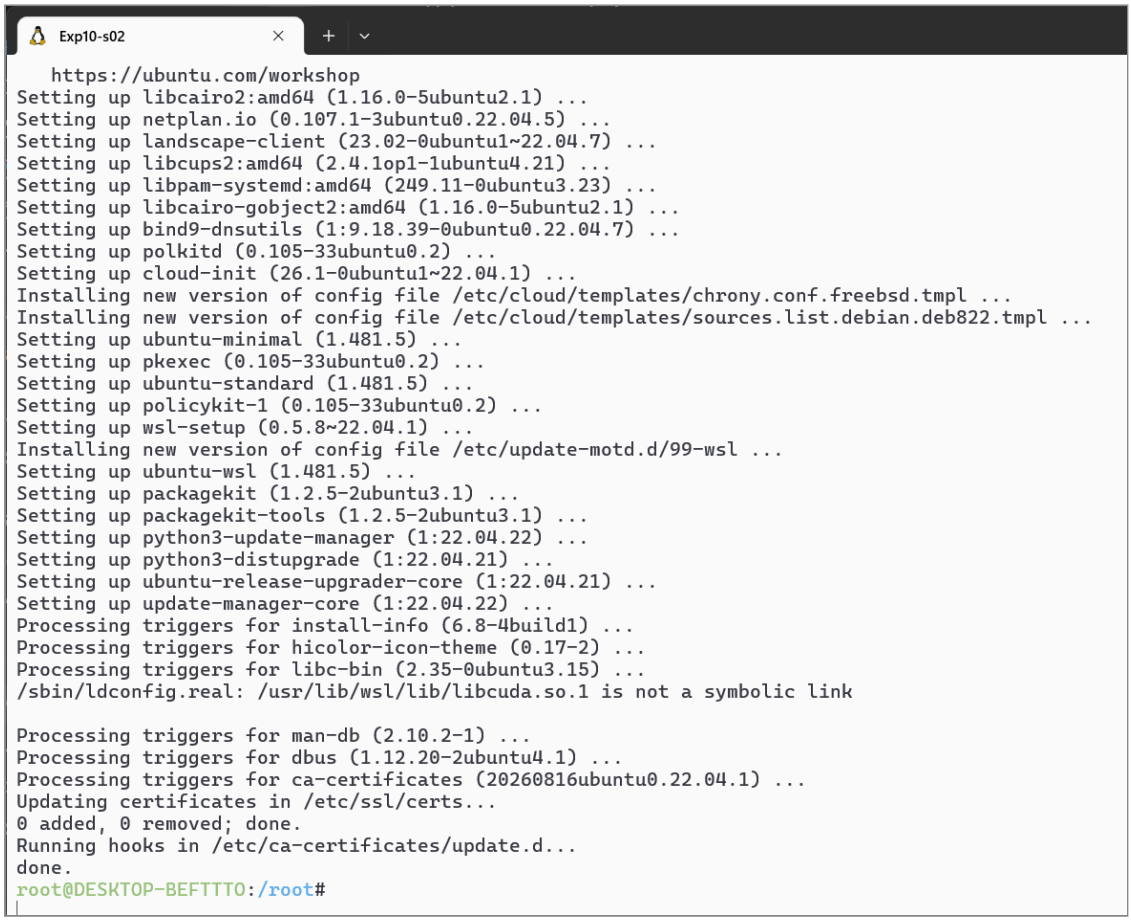


Fig. 2: sudo apt upgrade -y – 192 packages upgraded, 3 newly installed

Explanation:

This step updates the package index and upgrades all installed packages to their latest versions, so the system has current security patches and compatible libraries for Hyperledger Fabric. 192 packages were upgraded, including the Docker engine (docker.io), which avoids dependency conflicts later.

Step 2: Install Required Dependencies

Command:

```
sudo apt install jq curl git -y
node -v
npm -v
git --version
jq --version
docker --version
docker ps
```

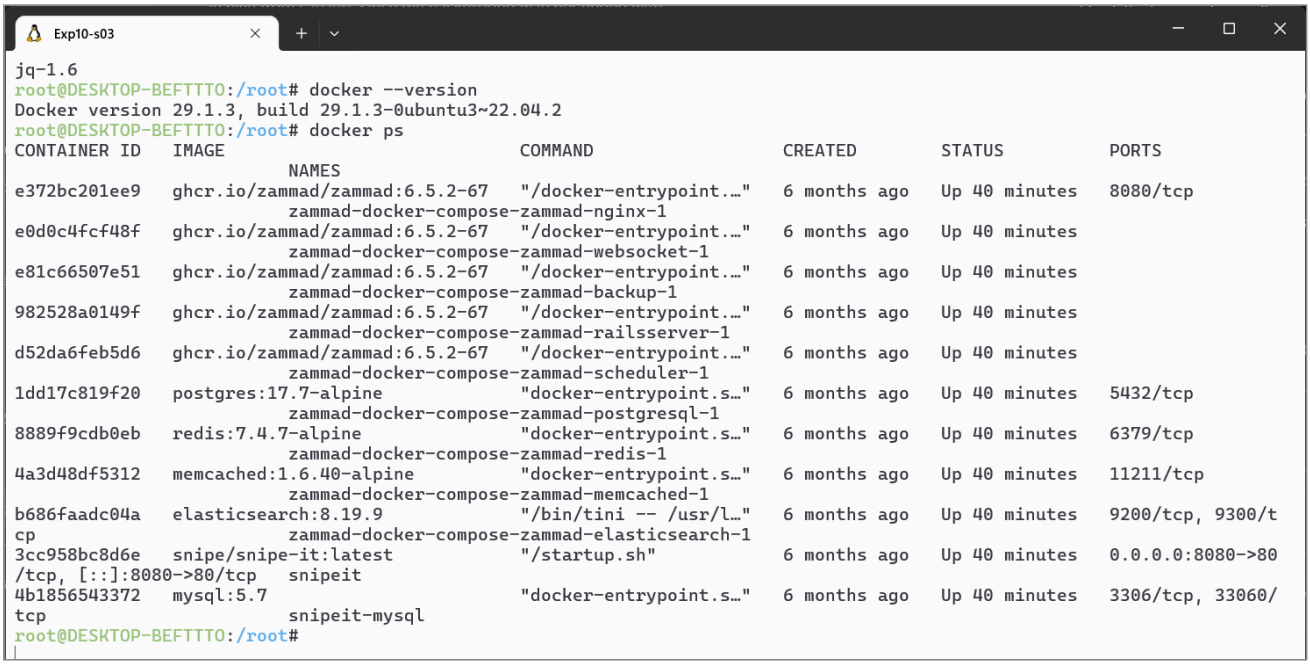


Fig. 3: Dependency versions – Node v18.20.8, npm 10.8.2, Git 2.34.1, jq 1.6, Docker 29.1.3; docker ps shows the daemon is running

Explanation:

This step installs and verifies the tools Hyperledger Fabric needs. Docker runs the blockchain components as containers, Git clones the repositories, jq processes JSON in the network scripts, and Node.js with npm runs the JavaScript chaincode. Node.js 18 and npm were already installed from NodeSource, so the Ubuntu *nodejs/npm* packages were not reinstalled (they would conflict with the NodeSource build). `docker ps` confirms the Docker daemon is running.

Step 3: Clone Fabric Samples

Command:

```
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
```

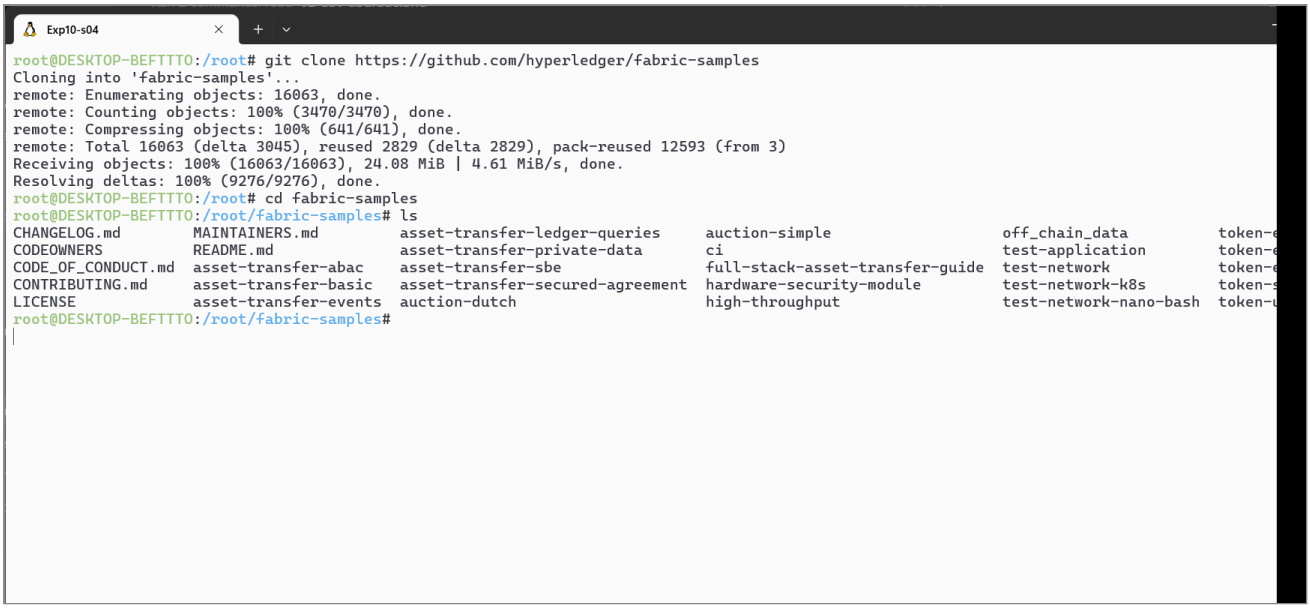


Fig. 4: fabric-samples cloned (16,063 objects) and its contents listed

Explanation:

This step downloads the official Hyperledger Fabric sample repository. It contains the *test-network* scripts and configuration, and example chaincodes such as *asset-transfer-basic*.

Step 4: Install Fabric Binaries and Images

Command:

```
curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh binary        # retry of the fabric-ca download
./install-fabric.sh docker
```

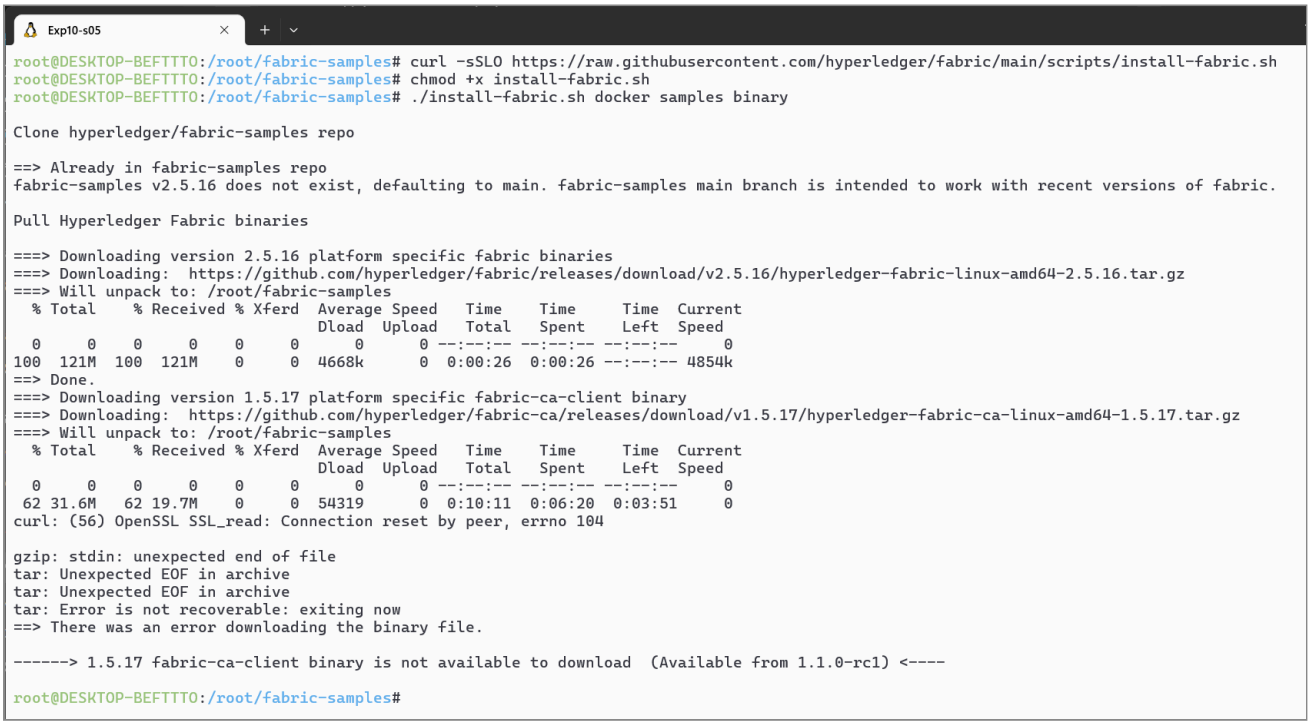


Fig. 5: install-fabric.sh – Fabric 2.5.16 binaries downloaded (121 MB); the fabric-ca 1.5.17 download was interrupted by the network ("Connection reset by peer")

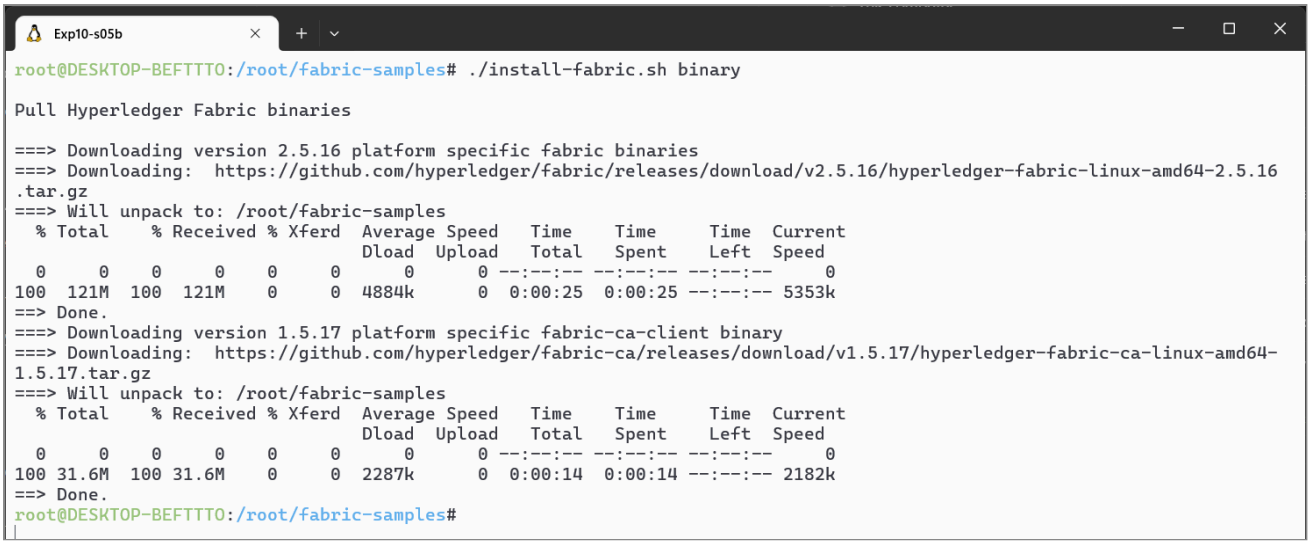


Fig. 6: Re-running ./install-fabric.sh binary – fabric-ca-client 1.5.17 downloaded completely ("Done")



Fig. 7: ./install-fabric.sh docker – peer, orderer, ccenv, baseos and fabric-ca images pulled and listed

Explanation:

This step downloads and runs the official install script. It installs the Fabric command-line binaries (peer, orderer, configtxgen, configtxlator, cryptogen, osnadmin, fabric-ca-client/server) into `fabric-samples/bin` and pulls the Docker images for the peer, orderer, chaincode environment and certificate authority. The first fabric-ca download broke off because the network connection was reset, so the binary download was repeated and the images were pulled separately.

Step 5: Verify Docker Images

Command:

```
docker images | grep fabric
```

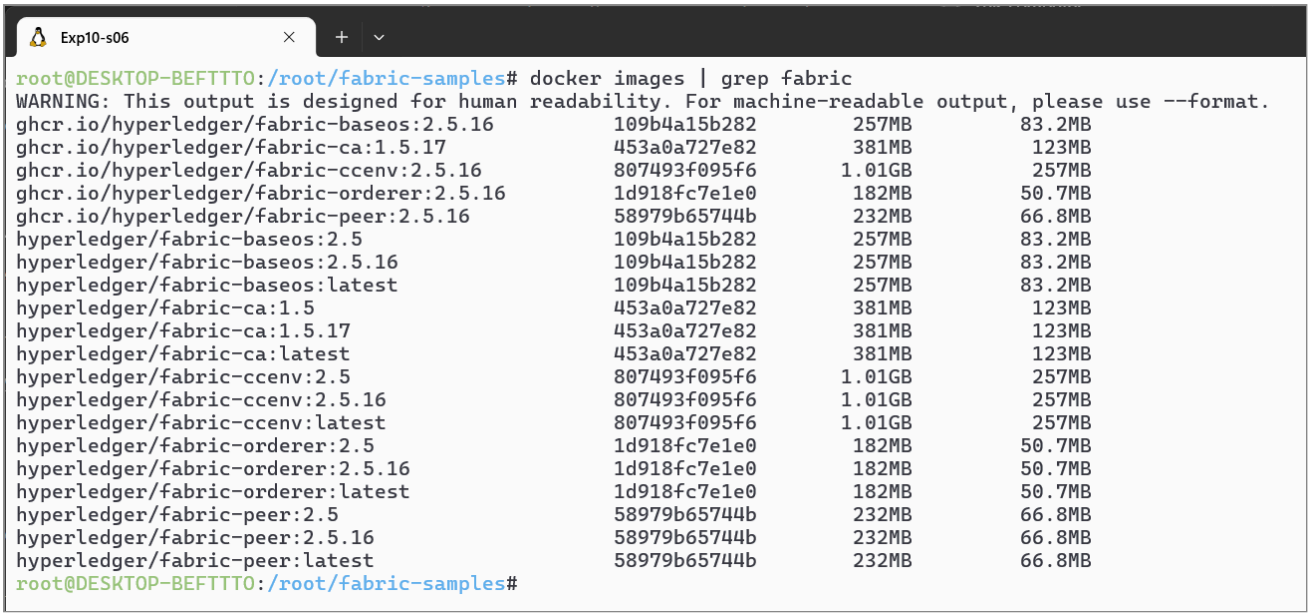


Fig. 8: Hyperledger Fabric 2.5.16 / CA 1.5.17 Docker images available locally

Explanation:

This command checks that all required Fabric Docker images were downloaded: fabric-peer, fabric-orderer, fabric-ccenv and fabric-baseos 2.5.16, and fabric-ca 1.5.17, each tagged 2.5 / 2.5.16 / latest. Missing images would cause deployment errors.

Step 6: Set Environment Variables

Command:

```
export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version
```



Fig. 9: peer version – v2.5.16, commit f871cf9, linux/amd64

Explanation:

This step sets the environment variables the Fabric CLI needs. PATH gives access to the Fabric binaries and FABRIC_CFG_PATH points to the core.yaml configuration. `peer version` (v2.5.16) confirms the setup works.

Step 7: Start Network and Create Channel

Command:

```
cd ~/fabric-samples/test-network
./network.sh up createChannel
```

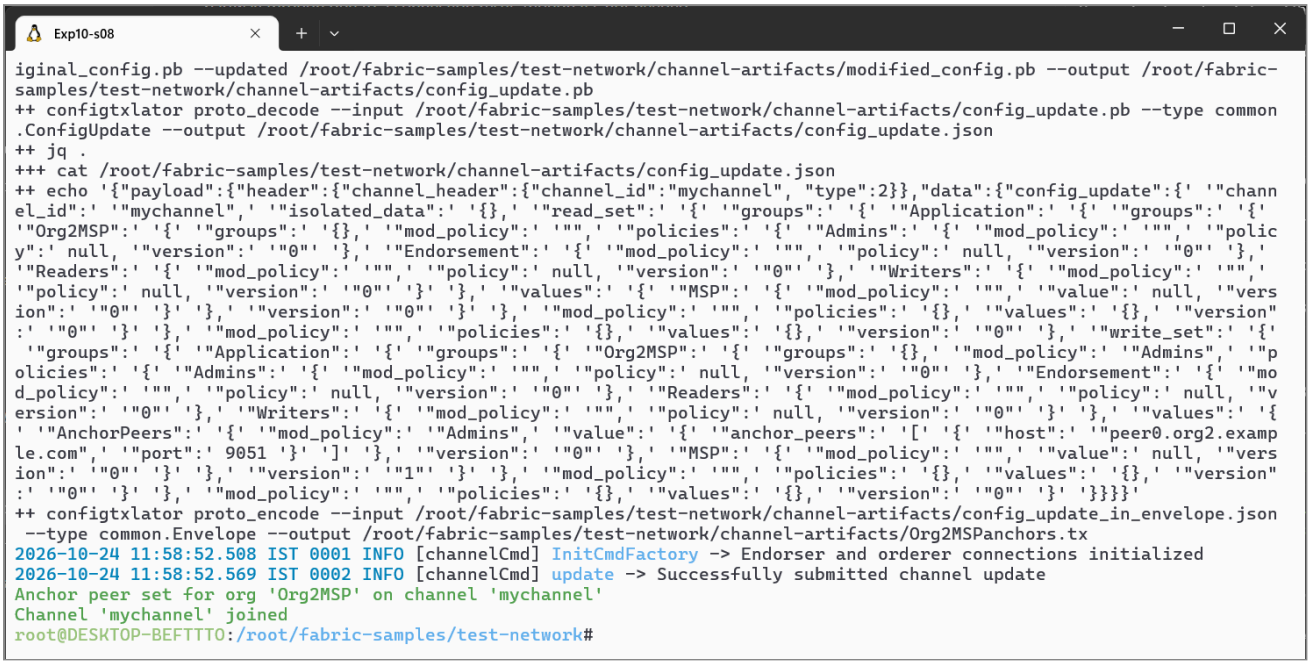


Fig. 10: network.sh up createChannel – anchor peers set for Org1 and Org2, "Channel 'mychannel' joined"

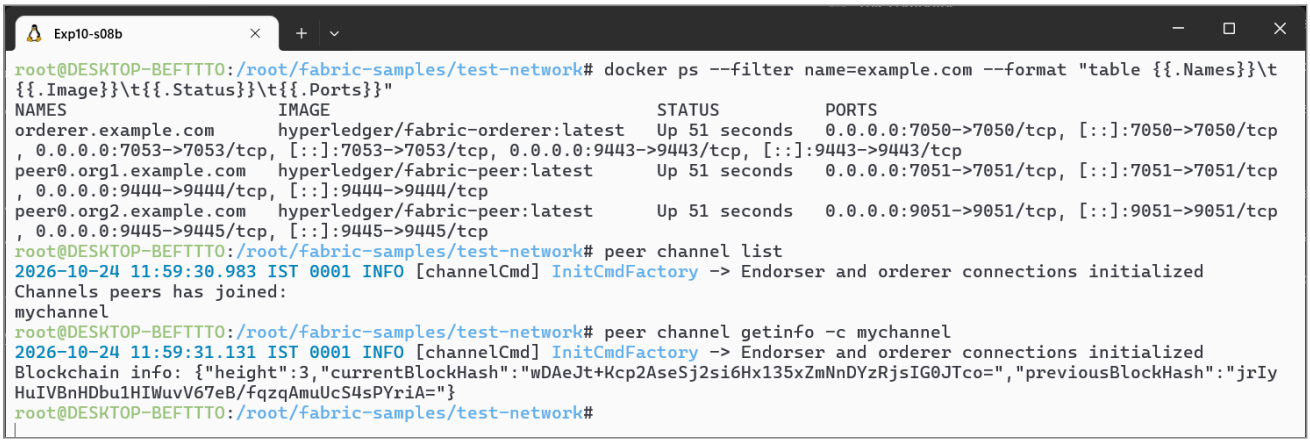


Fig. 11: Running nodes (orderer, peer0.org1, peer0.org2), peer channel list (mychannel) and channel height 3

Explanation:

This step starts the blockchain network. Certificates for Org1, Org2 and the orderer are generated with cryptogen, and the Docker containers for the orderer and the two peers are started. The channel *mychannel* is created, both peers join it, and the anchor peers are set. `peer channel list` and `getinfo` confirm the peer has joined *mychannel* (block height 3).

Step 8: Deploy Chaincode

Command:

```
./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript -ccl javascript
```

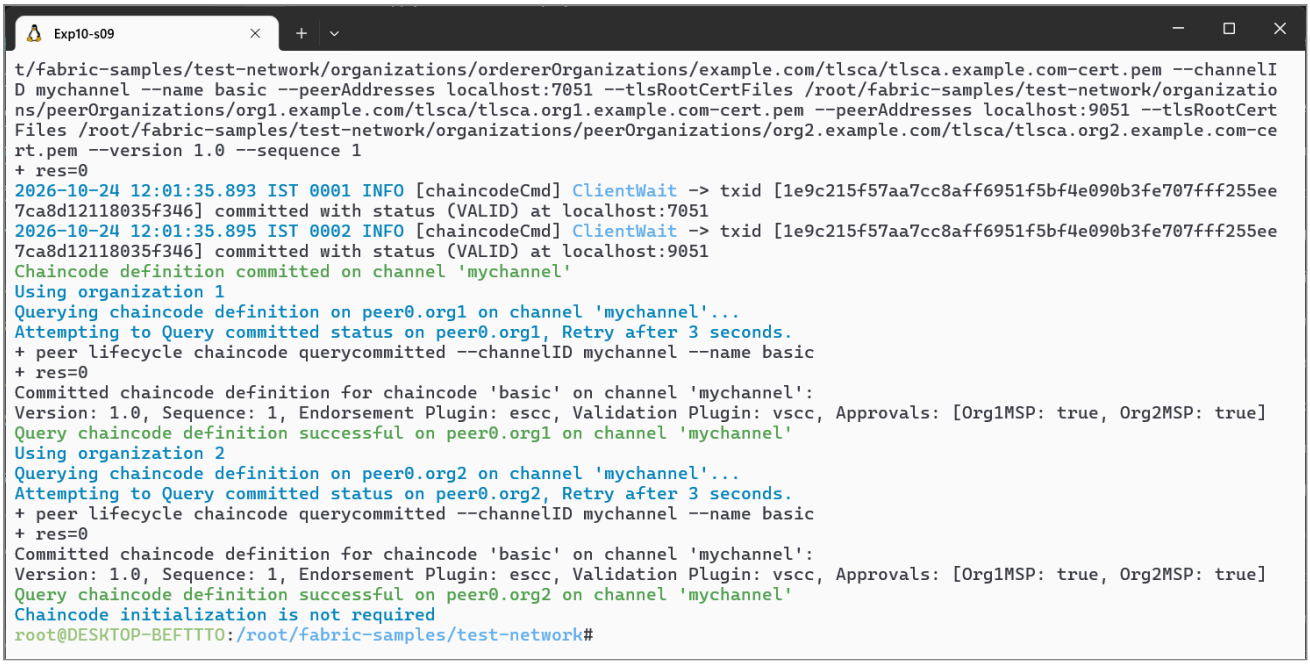


Fig. 12: Chaincode 'basic' v1.0, sequence 1, committed on mychannel with approvals from Org1MSP and Org2MSP

Explanation:

This step deploys the smart contract (chaincode) onto the network using the Fabric lifecycle. The chaincode is packaged (basic_1.0), installed on both peers, approved by Org1 and Org2, and its definition is committed to the channel. The query shows *Version 1.0, Sequence 1, Approvals: [Org1MSP: true, Org2MSP: true]*, so the chaincode is ready to use.

Step 9: Set Peer Environment

Command:

```
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganiz
ations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizatio
ns/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

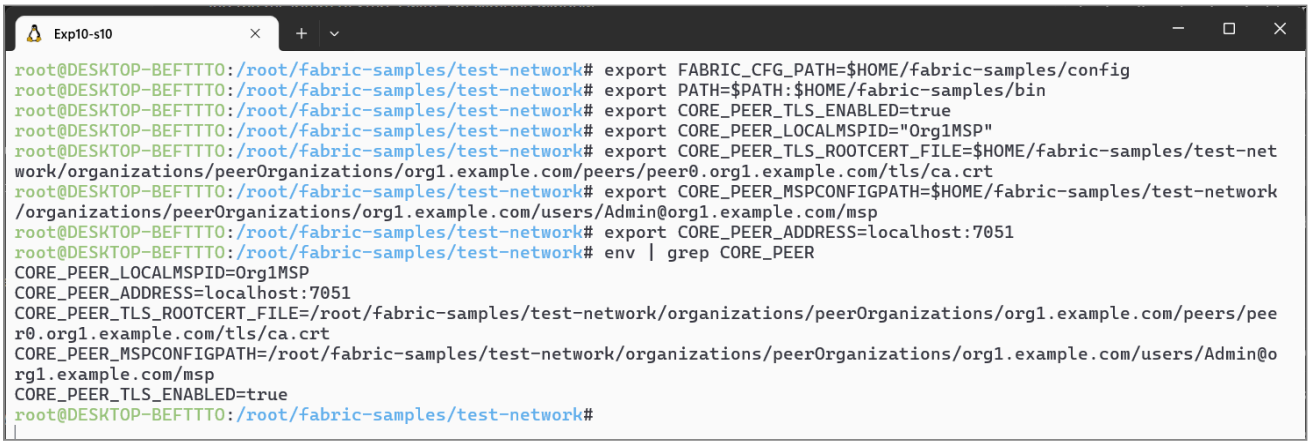


Fig. 13: Peer environment set to act as the Org1 admin on peer0.org1 (localhost:7051)

Explanation:

This step configures the identity and connection settings of the CLI. It acts as the Org1 Admin (Org1MSP) through peer0.org1 with TLS enabled, and specifies the TLS root certificate and MSP directory. These settings are needed for secure, authenticated interaction with the network.

Step 10: Initialize Ledger

Command:

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls
\
  --cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/or
derers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel -n basic \
  --peerAddresses localhost:7051 --tlsRootCertFiles $HOME/fabric-samples/test-network/organiza
tions/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 --tlsRootCertFiles $HOME/fabric-samples/test-network/organiza
tions/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
  --waitForEvent -c '{"function":"InitLedger","Args":[]}'
```



Fig. 14: InitLedger – txid 7134ed6f... committed with status (VALID) on both peers; Chaincode invoke successful (status 200)

Explanation:

This step initialises the ledger with six sample assets. It is a transaction: it is endorsed by peers of both organisations, ordered and committed. The transaction *7134ed6f976e7409...* was committed with status VALID at localhost:7051 and localhost:9051.

Step 11: Query All Assets

Command:

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```



Fig. 15: GetAllAssets – asset1 to asset6 returned from the world state

ID	Color	Size	Owner	AppraisedValue
asset1	blue	5	Tomoko	300
asset2	red	5	Brad	400

asset3	green	10	Jin Soo	500
asset4	yellow	10	Max	600
asset5	black	15	Adriana	700
asset6	white	15	Michel	800

Explanation:

This step retrieves all assets stored in the ledger. It is a read-only query: it shows the current data without creating a transaction, and confirms the ledger was initialised correctly.

Step 12: Read Specific Asset

Command:

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
```



Fig. 16: ReadAsset asset5 – black, size 15, owner Adriana, value 700

Explanation:

This step retrieves the details of a single asset by its ID, showing how individual records are read from the ledger. It is also a read-only operation.

Step 13: Create New Asset

Command:

```
peer chaincode invoke ... -c '{"function":"CreateAsset","Args":["asset10","blue","10","Khushi","1000"]}'
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
```



Fig. 17: CreateAsset asset10 (Owner: Khushi) – committed (VALID); ReadAsset confirms the new asset

Explanation:

This step creates a new asset on the blockchain with ID asset10, colour blue, size 10, owner **Khushi** and value 1000. The owner name confirms that the transaction was performed by the student. The transaction *6869175eee9a8aaf...* was committed with status VALID on both peers, and ReadAsset returns the stored asset.

Step 14: Transfer Asset

Command:

```
peer chaincode invoke ... -c '{"function":"TransferAsset","Args":["asset10","Harsh"]}'
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
```



Fig. 18: TransferAsset asset10 to Harsh – payload "Khushi" (previous owner); ReadAsset now shows Owner "Harsh"

Explanation:

This step transfers ownership of asset10 from Khushi to Harsh by updating the Owner field in the ledger. The chaincode returns the previous owner ("Khushi") as the payload. The transaction *35baae393bb00da4…* was committed as VALID, and a new ReadAsset shows *"Owner":"Harsh"*. This shows how a blockchain keeps asset transfers secure and traceable.

Step 15: Stop Network

Command:

```
./network.sh down
```

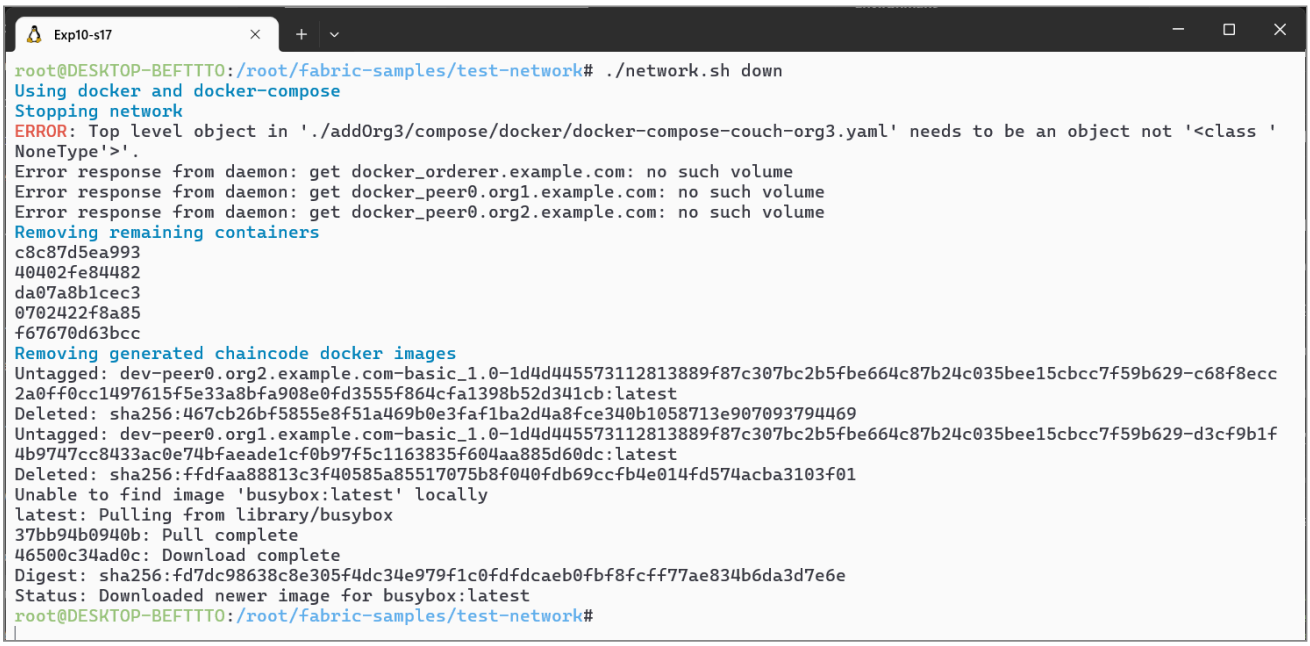


Fig. 19: network.sh down – Fabric containers, volumes and generated chaincode images removed

Explanation:

This step stops and removes the Fabric containers (orderer, peers and chaincode containers), their volumes and the generated chaincode Docker images, then cleans up the generated certificates. It

frees system resources and safely ends the experiment.

Result:

The Hyperledger Fabric 2.5.16 test network (one Raft orderer and two peers from Org1 and Org2) was implemented on Ubuntu 22.04 under WSL 2 with Docker. The channel *mychannel* was created and the JavaScript chaincode *basic* (v1.0) was deployed with approvals from both organisations. Asset operations were then performed successfully:

Operation	Type	Outcome
InitLedger	invoke	txid 7134ed6f… VALID – 6 assets created
GetAllAssets	query	asset1 – asset6 returned
ReadAsset asset5	query	Owner Adriana, value 700
CreateAsset asset10	invoke	txid 6869175e… VALID – Owner **Khushi**, value 1000
TransferAsset asset10 → Harsh	invoke	txid 35baae39… VALID – Owner changed Khushi → Harsh

The asset was created with owner name **Khushi**, confirming correct execution.

Conclusion:

This experiment showed how a permissioned blockchain network works in Hyperledger Fabric. Peers, an ordering service and a channel were set up as Docker containers, and chaincode was deployed through the package–install–approve–commit lifecycle. Each state-changing transaction (InitLedger, CreateAsset, TransferAsset) was endorsed by both organisations, ordered and committed as VALID, while queries read the world state without creating transactions. The experiment also demonstrated real-time asset management (creating, querying and transferring ownership) using smart contracts.